

WebGL Rendering Context Specification & Architecture Reference

Standard: WebGL 1.0 / OpenGL ES 2.0 Web IDL Specification Interface

Exposed: [Exposed=(Window,Worker)]

Table of Contents

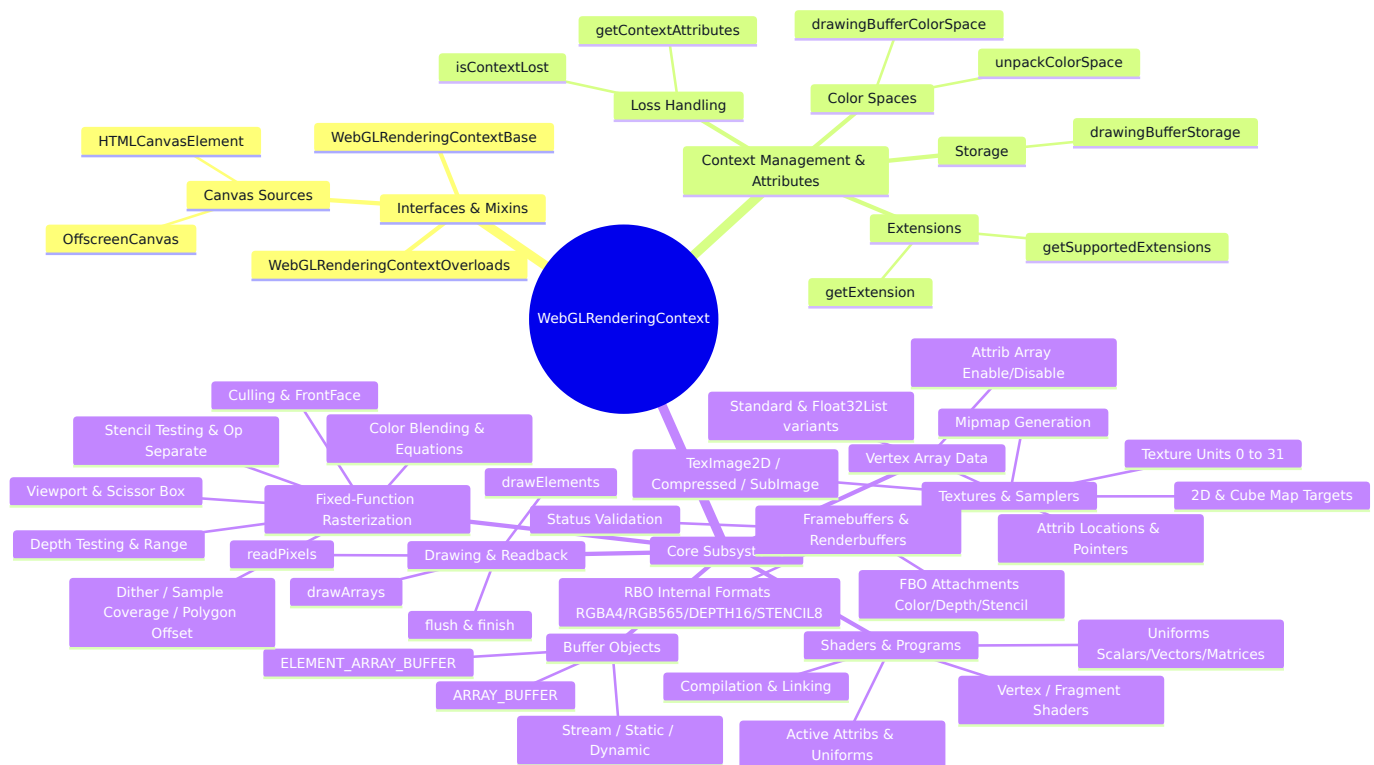
1. [Architectural Overview & Mind Maps](#)
 - [High-Level Mind Map](#)
 - [WebGL State Machine & Pipeline Flow](#)
 - [Object Hierarchy & Bindings](#)
2. [Interface Declarations & Typedefs](#)
 - [Typedefs](#)
 - [Interface Hierarchy & Mixins](#)
 - [Context Attributes & Lifecycle Properties](#)
3. [Exhaustive Enum Reference \(Categorized & Tabulated\)](#)
 - [3.1 ClearBufferMask](#)
 - [3.2 BeginMode \(Primitive Topologies\)](#)
 - [3.3 AlphaFunction \(Legacy / Not supported in ES20\)](#)
 - [3.4 Blending Factors & Equations](#)
 - [3.5 Buffer Objects & Usage Patterns](#)
 - [3.6 Cull Face & Front Face Modes](#)
 - [3.7 Depth & Stencil Comparison Functions](#)
 - [3.8 Stencil Operations & Back Stencil State](#)
 - [3.9 Enable Capabilities \(EnableCap\)](#)
 - [3.10 Error Codes](#)
 - [3.11 Parameter Queries \(GetPName\)](#)
 - [3.12 Hints \(HintTarget & HintMode\)](#)
 - [3.13 Data Types \(DataType\)](#)
 - [3.14 Pixel Formats & Pixel Types](#)
 - [3.15 Shaders, Programs & Precision Types](#)
 - [3.16 Uniform Types](#)
 - [3.17 Vertex Attributes](#)
 - [3.18 Textures \(Targets, Units, Filters, Wrap Modes & Parameters\)](#)
 - [3.19 Framebuffers & Renderbuffers](#)
 - [3.20 Read Format Queries](#)
 - [3.21 Implementation Strings \(StringName\)](#)
 - [3.22 WebGL-Specific Constants](#)

4. Exhaustive Function Reference

- [4.1 Context Lifecycle, Capabilities & Extensions](#)
- [4.2 Frame Clearing & Viewport / Scissor Rectangles](#)
- [4.3 Rasterizer State & Depth / Stencil / Color Masking](#)
- [4.4 Blending Configuration](#)
- [4.5 Buffer Management & Data Upload](#)
- [4.6 Shaders & Programs Pipeline](#)
- [4.7 Vertex Attributes & Pointers](#)
- [4.8 Uniform State \(Scalars, Vectors, Matrices\)](#)
- [4.9 Textures & Samplers Management](#)
- [4.10 Framebuffer & Renderbuffer Subsystem](#)
- [4.11 Drawing Primitives & Execution Control](#)
- [4.12 Pixel Readback & Storage Modes](#)

1. Architectural Overview & Mind Maps

1.1 High-Level Mind Map



1.1.1 Subsystem Navigation Map

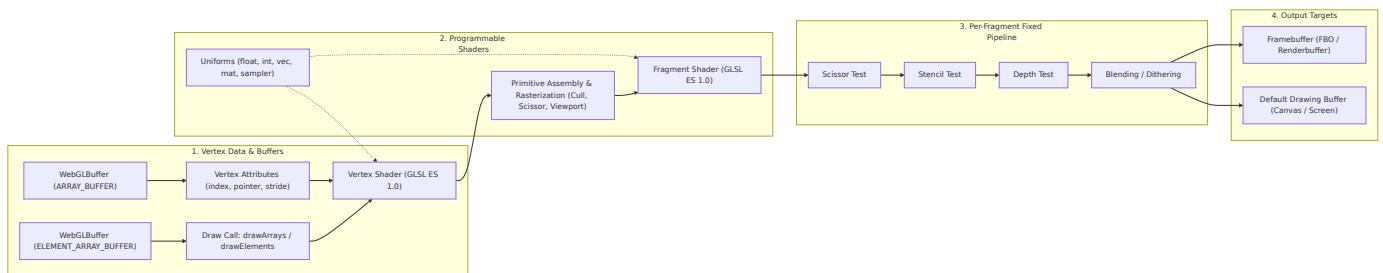
Jump to any subsystem, function, or enum section across the documentation:

- [Context Management & Lifecycle](#)
 - Lifecycle & Status: [isContextLost\(\)](#) | [getContextAttributes\(\)](#) | [drawingBufferStorage\(\)](#)
 - Extensions: [getExtension\(\)](#) | [getSupportedExtensions\(\)](#)
 - Context Attributes: [canvas](#) | [drawingBufferWidth](#) | [drawingBufferHeight](#) | [drawingBufferFormat](#) | [drawingBufferColorSpace](#) | [unpackColorSpace](#)

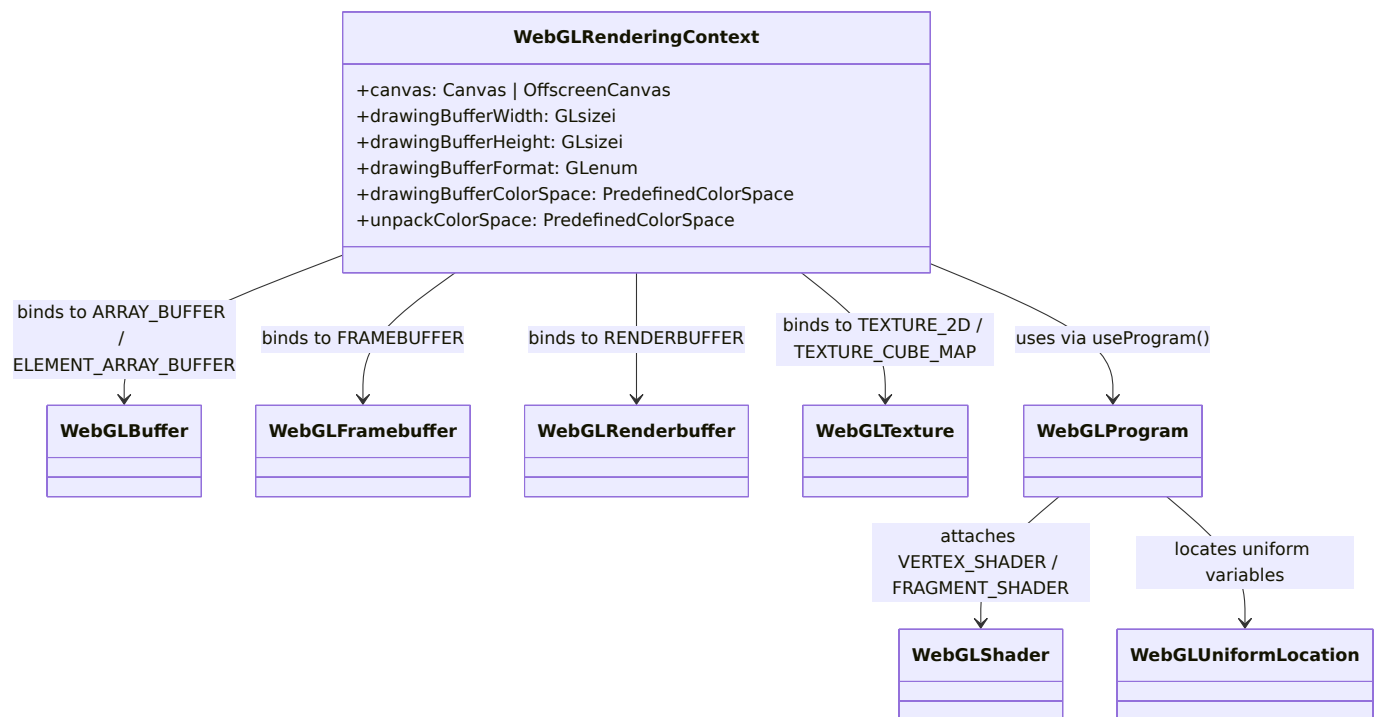
- Error & Query: [getError\(\)](#) | [getParameter\(\)](#) | [hint\(\)](#)
- Capabilities: [enable\(\)](#) | [disable\(\)](#) | [isEnabled\(\)](#)
-  **Buffer Objects & Vertex Geometry**
 - Targets: [ARRAY_BUFFER](#) | [ELEMENT_ARRAY_BUFFER](#)
 - Usage Hints: [STATIC_DRAW](#) | [DYNAMIC_DRAW](#) | [STREAM_DRAW](#)
 - Management: [createBuffer\(\)](#) | [bindBuffer\(\)](#) | [bufferData\(\)](#) | [bufferSubData\(\)](#) | [deleteBuffer\(\)](#) | [isBuffer\(\)](#) | [getBufferParameter\(\)](#)
 - Attributes: [vertexAttribPointer\(\)](#) | [enableVertexAttribArray\(\)](#) | [disableVertexAttribArray\(\)](#) | [bindAttribLocation\(\)](#) | [getAttribLocation\(\)](#) | [getActiveAttrib\(\)](#) | [getVertexAttrib\(\)](#) | [getVertexAttribOffset\(\)](#)
 - Generic Values: [vertexAttrib1f .. 4f](#) | [vertexAttrib1fv .. 4fv](#)
-  **Shaders & GPU Programs**
 - Stages: [VERTEX_SHADER](#) | [FRAGMENT_SHADER](#)
 - Shader Lifecycle: [createShader\(\)](#) | [shaderSource\(\)](#) | [compileShader\(\)](#) | [getShaderParameter\(\)](#) | [getShaderInfoLog\(\)](#) | [getShaderSource\(\)](#) | [getShaderPrecisionFormat\(\)](#) | [deleteShader\(\)](#) | [isShader\(\)](#)
 - Program Lifecycle: [createProgram\(\)](#) | [attachShader\(\)](#) | [detachShader\(\)](#) | [linkProgram\(\)](#) | [useProgram\(\)](#) | [validateProgram\(\)](#) | [getProgramParameter\(\)](#) | [getProgramInfoLog\(\)](#) | [getAttachedShaders\(\)](#) | [deleteProgram\(\)](#) | [isProgram\(\)](#)
 - Uniforms: [getUniformLocation\(\)](#) | [getActiveUniform\(\)](#) | [getUniform\(\)](#) | [uniform1f .. 4f](#) | [uniform1i .. 4i](#) | [uniform1fv .. 4fv](#) | [uniform1iv .. 4iv](#) | [uniformMatrix2fv .. 4fv](#)
-  **Textures & Samplers**
 - Targets: [TEXTURE_2D](#) | [TEXTURE_CUBE_MAP](#) (+X, -X, +Y, -Y, +Z, -Z)
 - Units: [activeTexture\(\)](#) with [TEXTURE0 .. TEXTURE31](#)
 - Upload: [texImage2D\(\)](#) | [texSubImage2D\(\)](#) | [copyTexImage2D\(\)](#) | [copyTexSubImage2D\(\)](#) | [compressedTexImage2D\(\)](#) | [compressedTexSubImage2D\(\)](#)
 - Filters & Wrap: [texParameteri\(\)](#) | [texParameterf\(\)](#) | [getTexParameter\(\)](#) | [generateMipmap\(\)](#) | [pixelStorei\(\)](#)
 - Lifecycle: [createTexture\(\)](#) | [bindTexture\(\)](#) | [deleteTexture\(\)](#) | [isTexture\(\)](#)
-  **Rasterization & Fixed-Function State**
 - Geometry: [viewport\(\)](#) | [scissor\(\)](#) | [cullFace\(\)](#) | [frontFace\(\)](#) | [lineWidth\(\)](#) | [polygonOffset\(\)](#)
 - Clearing: [clear\(\)](#) | [clearColor\(\)](#) | [clearDepth\(\)](#) | [clearStencil\(\)](#)
 - Testing & Masks: [depthFunc\(\)](#) | [depthRange\(\)](#) | [depthMask\(\)](#) | [colorMask\(\)](#) | [stencilFunc\(\)](#) | [stencilFuncSeparate\(\)](#) | [stencilMask\(\)](#) | [stencilMaskSeparate\(\)](#) | [stencilOp\(\)](#) | [stencilOpSeparate\(\)](#) | [sampleCoverage\(\)](#)
 - Blending: [blendColor\(\)](#) | [blendEquation\(\)](#) | [blendEquationSeparate\(\)](#) | [blendFunc\(\)](#) | [blendFuncSeparate\(\)](#)
-  **Framebuffer & Renderbuffer (FBO/RBO)**
 - Targets: [FRAMEBUFFER](#) | [RENDERBUFFER](#)

- Framebuffer: [createFramebuffer\(\)](#) | [bindFramebuffer\(\)](#) | [framebufferTexture2D\(\)](#) | [framebufferRenderbuffer\(\)](#) | [checkFramebufferStatus\(\)](#) | [getFramebufferAttachmentParameter\(\)](#) | [deleteFramebuffer\(\)](#) | [isFramebuffer\(\)](#)
- Renderbuffer: [createRenderbuffer\(\)](#) | [bindRenderbuffer\(\)](#) | [renderbufferStorage\(\)](#) | [getRenderbufferParameter\(\)](#) | [deleteRenderbuffer\(\)](#) | [isRenderbuffer\(\)](#)
- 🎨 **Drawing, Execution & Readback**
 - Drawing: [drawArrays\(\)](#) | [drawElements\(\)](#)
 - Execution: [flush\(\)](#) | [finish\(\)](#)
 - Readback: [readPixels\(\)](#)

1.2 WebGL State Machine & Pipeline Flow



1.3 Object Hierarchy & Bindings



2. Interface Declarations & Typedefs

2.1 Typedefs

The IDL specifies three core typedefs defining acceptable image sources and numerical array types:

Typedef Name	Type Definition	Description / Purpose
--------------	-----------------	-----------------------

<code>TexImageSource</code>	<code>ImageBitmap</code> <code>ImageData</code> <code>HTMLImageElement</code> <code>HTMLCanvasElement</code> <code>HTMLVideoElement</code> <code>OffscreenCanvas</code> <code>VideoFrame</code>	Poly-type representing pixel/image input sources usable directly in <code>texImage2D</code> and <code>texSubImage2D</code> .
<code>Float32List</code>	<code>([AllowShared] Float32Array or sequence<GLfloat>)</code>	List of single-precision IEEE 754 float values, permitting typed array views (including shared buffers) or standard WebIDL sequences.
<code>Int32List</code>	<code>([AllowShared] Int32Array or sequence<GLint>)</code>	List of 32-bit signed integers, permitting typed array views (including shared buffers) or standard WebIDL sequences.

2.2 Interface Hierarchy & Mixins

```
[Exposed=(Window,Worker)]
interface WebGLRenderingContext {
};
WebGLRenderingContext includes WebGLRenderingContextBase;
WebGLRenderingContext includes WebGLRenderingContextOverloads;
```

- **Exposure:** Exposed in both standard browsing context (`Window`) and web background workers (`Worker`).
- **Composition:** Combines `WebGLRenderingContextBase` (core OpenGL ES 2.0 WebIDL binding) and `WebGLRenderingContextOverloads` (overloaded buffer and texture methods supporting JavaScript-specific types like `TypedArrays` and `TexImageSource`).

2.3 Context Attributes & Lifecycle Properties

Context Attributes

- **`canvas`** (readonly attribute (`HTMLCanvasElement` or `OffscreenCanvas`) `canvas`;)

Returns a reference to the source canvas or offscreen canvas that owns this WebGL context.
- **`drawingBufferWidth`** (readonly attribute `GLsizei` `drawingBufferWidth`;)

The actual width of the drawing buffer (in pixels), which might differ from canvas width due to device pixel ratios or memory constraints.
- **`drawingBufferHeight`** (readonly attribute `GLsizei` `drawingBufferHeight`;)

The actual height of the drawing buffer (in pixels).
- **`drawingBufferFormat`** (readonly attribute `GLenum` `drawingBufferFormat`;)

Returns the internal format of the drawing buffer (e.g., `RGBA`, `RGB`).
- **`drawingBufferColorSpace`** (attribute `PredefinedColorSpace` `drawingBufferColorSpace`;)

Color space of the rendering context drawing buffer. Defaults to `"srgb"`.
- **`unpackColorSpace`** (attribute `PredefinedColorSpace` `unpackColorSpace`;)

Color space converted to when unpacking pixel data into textures. Defaults to `"srgb"`.

3. Exhaustive Enum Reference (Categorized & Tabulated)

3.1 `ClearBufferMask`

Mask bits passed to `clear()` to selectively clear specific render buffers.

Constant Name	Hex Value	Decimal	Description
DEPTH_BUFFER_BIT	0x00000100	256	Bit flag to clear the depth buffer to <code>clearDepth</code> .
STENCIL_BUFFER_BIT	0x00000400	1024	Bit flag to clear the stencil buffer to <code>clearStencil</code> .
COLOR_BUFFER_BIT	0x00004000	16384	Bit flag to clear the color buffer to <code>clearColor</code> .

3.2 BeginMode (Primitive Topologies)

Primitive assembly modes passed to `drawArrays()` and `drawElements()`.

Constant Name	Hex Value	Decimal	Description
POINTS	0x0000	0	Draws individual points per vertex.
LINES	0x0001	1	Draws a series of unconnected line segments (v0-v1, v2-v3, ...).
LINE_LOOP	0x0002	2	Draws connected line segments, connecting the last vertex back to the first.
LINE_STRIP	0x0003	3	Draws connected line segments (v0-v1, v1-v2, v2-v3, ...).
TRIANGLES	0x0004	4	Draws independent triangles for every 3 vertices (v0-v1-v2, v3-v4-v5, ...).
TRIANGLE_STRIP	0x0005	5	Draws linked triangles sharing two consecutive vertices.
TRIANGLE_FAN	0x0006	6	Draws linked triangles sharing the first root vertex (v0-v1-v2, v0-v2-v3, ...).

3.3 AlphaFunction (Legacy / Not supported in ES20)

*Note from specification: Listed in legacy OpenGL definitions, but **not supported in ES20 / WebGL 1.0**.*

Constant Name	Status	Comparison Meaning
NEVER	Not supported in ES20	Never passes test.
LESS	Not supported in ES20	Passes if incoming alpha < reference alpha.
EQUAL	Not supported in ES20	Passes if incoming alpha == reference alpha.
LEQUAL	Not supported in ES20	Passes if incoming alpha <= reference alpha.
GREATER	Not supported in ES20	Passes if incoming alpha > reference alpha.
NOTEQUAL	Not supported in ES20	Passes if incoming alpha != reference alpha.
GEQUAL	Not supported in ES20	Passes if incoming alpha >= reference alpha.
ALWAYS	Not supported in ES20	Always passes test.

3.4 Blending Factors & Equations

BlendingFactorDest & BlendingFactorSrc

Constants used in `blendFunc()` and `blendFuncSeparate()` .

Constant Name	Hex Value	Decimal	Source Factor	Destination Factor	Formula / Meaning
ZERO	0	0	Yes	Yes	(0, 0, 0, 0)
ONE	1	1	Yes	Yes	(1, 1, 1, 1)
SRC_COLOR	0x0300	768	—	Yes	(Rs, Gs, Bs, As)
ONE_MINUS_SRC_COLOR	0x0301	769	—	Yes	(1-Rs, 1-Gs, 1-Bs, 1-As)
SRC_ALPHA	0x0302	770	Yes	Yes	(As, As, As, As)
ONE_MINUS_SRC_ALPHA	0x0303	771	Yes	Yes	(1-As, 1-As, 1-As, 1-As)
DST_ALPHA	0x0304	772	Yes	Yes	(Ad, Ad, Ad, Ad)
ONE_MINUS_DST_ALPHA	0x0305	773	Yes	Yes	(1-Ad, 1-Ad, 1-Ad, 1-Ad)
DST_COLOR	0x0306	774	Yes	—	(Rd, Gd, Bd, Ad)
ONE_MINUS_DST_COLOR	0x0307	775	Yes	—	(1-Rd, 1-Gd, 1-Bd, 1-Ad)
SRC_ALPHA_SATURATE	0x0308	776	Yes	—	(f, f, f, 1) where f = min(As, 1 - Ad)
CONSTANT_COLOR	0x8001	32769	Yes	Yes	(Rc, Gc, Bc, Ac) (set via <code>blendColor</code>)
ONE_MINUS_CONSTANT_COLOR	0x8002	32770	Yes	Yes	(1-Rc, 1-Gc, 1-Bc, 1-Ac)
CONSTANT_ALPHA	0x8003	32771	Yes	Yes	(Ac, Ac, Ac, Ac)
ONE_MINUS_CONSTANT_ALPHA	0x8004	32772	Yes	Yes	(1-Ac, 1-Ac, 1-Ac, 1-Ac)

BlendEquation & Separate Blend Functions

Constants used in `blendEquation()` , `blendEquationSeparate()` , and parameter queries.

Constant Name	Hex Value	Decimal	Description / Equation
FUNC_ADD	0x8006	32774	Result = Src * S + Dst * D
BLEND_EQUATION	0x8009	32777	Query parameter for current blend equation mode.
BLEND_EQUATION_RGB	0x8009	32777	Alias to BLEND_EQUATION ; queries RGB blend equation.
BLEND_EQUATION_ALPHA	0x883D	34877	Query parameter for Alpha blend equation.
FUNC_SUBTRACT	0x800A	32778	Result = Src * S - Dst * D
FUNC_REVERSE_SUBTRACT	0x800B	32779	Result = Dst * D - Src * S
BLEND_DST_RGB	0x80C8	32968	Query parameter for destination RGB blend factor.

BLEND_SRC_RGB	0x80C9	32969	Query parameter for source RGB blend factor.
BLEND_DST_ALPHA	0x80CA	32970	Query parameter for destination Alpha blend factor.
BLEND_SRC_ALPHA	0x80CB	32971	Query parameter for source Alpha blend factor.
BLEND_COLOR	0x8005	32773	Query parameter for the constant blend color (Rc, Gc, Bc, Ac) .

3.5 Buffer Objects & Usage Patterns

Constant Name	Hex Value	Decimal	Category	Description
ARRAY_BUFFER	0x8892	34962	Target	Target for vertex attribute data (coordinates, normals, texcoords).
ELEMENT_ARRAY_BUFFER	0x8893	34963	Target	Target for vertex index arrays (used by drawElements).
ARRAY_BUFFER_BINDING	0x8894	34964	Query	Query currently bound ARRAY_BUFFER .
ELEMENT_ARRAY_BUFFER_BINDING	0x8895	34965	Query	Query currently bound ELEMENT_ARRAY_BUFFER .
STREAM_DRAW	0x88E0	35040	Usage	Buffer data modified once and used a few times for drawing.
STATIC_DRAW	0x88E4	35044	Usage	Buffer data modified once and used many times for drawing.
DYNAMIC_DRAW	0x88E8	35048	Usage	Buffer data modified repeatedly and used many times for drawing.
BUFFER_SIZE	0x8764	34660	Query	Queries buffer allocation size in bytes via getBufferParameter .
BUFFER_USAGE	0x8765	34661	Query	Queries usage hint (STATIC_DRAW , etc.) via getBufferParameter .
CURRENT_VERTEX_ATTRIB	0x8626	34342	Query	Queries current generic vertex attribute values.

3.6 Cull Face & Front Face Modes

Constant Name	Hex Value	Decimal	Classification	Description
FRONT	0x0404	1028	cullFace mode	Culls front-facing polygons.
BACK	0x0405	1029	cullFace mode	Culls back-facing polygons.
FRONT_AND_BACK	0x0408	1032	cullFace mode	Culls both front- and back-facing polygons.

CW	0x0900	2304	frontFace direction	Clockwise winding order designates front face.
CCW	0x0901	2305	frontFace direction	Counter-clockwise winding order designates front face.

3.7 Depth & Stencil Comparison Functions

Comparison functions used by `depthFunc()` , `stencilFunc()` , and `stencilFuncSeparate()` .

Constant Name	Hex Value	Decimal	Comparison Condition
NEVER	0x0200	512	Test never passes.
LESS	0x0201	513	Passes if incoming value < stored value.
EQUAL	0x0202	514	Passes if incoming value == stored value.
LEQUAL	0x0203	515	Passes if incoming value <= stored value.
GREATER	0x0204	516	Passes if incoming value > stored value.
NOTEQUAL	0x0205	517	Passes if incoming value != stored value.
GEQUAL	0x0206	518	Passes if incoming value >= stored value.
ALWAYS	0x0207	519	Test always passes.

3.8 Stencil Operations & Back Stencil State

StencilOp Actions

Used in `stencilOp()` and `stencilOpSeparate()` .

Constant Name	Hex Value	Decimal	Action on Stencil Buffer
ZERO	0	0	Sets stencil buffer value to 0.
KEEP	0x1E00	7680	Keeps current value unchanged.
REPLACE	0x1E01	7681	Sets stencil value to reference value specified by <code>stencilFunc</code> .
INCR	0x1E02	7682	Increments stencil value; clamps to maximum value.
DECR	0x1E03	7683	Decrements stencil value; clamps to 0.
INVERT	0x150A	5386	Bitwise-inverts current stencil value.
INCR_WRAP	0x8507	34055	Increments stencil value; wraps around to 0 on overflow.
DECR_WRAP	0x8508	34056	Decrements stencil value; wraps around to max on underflow.

Stencil State Queries (Front & Back)

Constant Name	Hex Value	Decimal	Description
---------------	-----------	---------	-------------

STENCIL_CLEAR_VALUE	0x0B91	2961	Clear value for stencil buffer.
STENCIL_FUNC	0x0B92	2962	Front stencil test function.
STENCIL_FAIL	0x0B94	2964	Front stencil fail action.
STENCIL_PASS_DEPTH_FAIL	0x0B95	2965	Front stencil pass, depth test fail action.
STENCIL_PASS_DEPTH_PASS	0x0B96	2966	Front stencil pass, depth test pass action.
STENCIL_REF	0x0B97	2967	Front stencil reference value.
STENCIL_VALUE_MASK	0x0B93	2963	Front stencil comparison mask.
STENCIL_WRITEMASK	0x0B98	2968	Front stencil write mask.
STENCIL_BACK_FUNC	0x8800	34816	Back stencil test function.
STENCIL_BACK_FAIL	0x8801	34817	Back stencil fail action.
STENCIL_BACK_PASS_DEPTH_FAIL	0x8802	34818	Back stencil pass, depth fail action.
STENCIL_BACK_PASS_DEPTH_PASS	0x8803	34819	Back stencil pass, depth pass action.
STENCIL_BACK_REF	0x8CA3	36003	Back stencil reference value.
STENCIL_BACK_VALUE_MASK	0x8CA4	36004	Back stencil comparison mask.
STENCIL_BACK_WRITEMASK	0x8CA5	36005	Back stencil write mask.

3.9 Enable Capabilities (EnableCap)

Flags toggled via `enable()`, `disable()`, and queried with `isEnabled()`.

Constant Name	Hex Value	Decimal	Capability Enabled When Set
CULL_FACE	0x0B44	2884	Polygon culling (culls front or back faces).
BLEND	0x0BE2	3042	Fragment color blending with drawing buffer.
DITHER	0x0BD0	3024	Dithering color components before writing to frame buffer.
STENCIL_TEST	0x0B90	2960	Stencil testing and updates to stencil buffer.
DEPTH_TEST	0x0B71	2929	Depth buffer comparison and depth buffer updates.
SCISSOR_TEST	0x0C11	3089	Discards fragments outside the scissor box rectangle.
POLYGON_OFFSET_FILL	0x8037	32823	Adds depth offset to fragments of polygon primitives.
SAMPLE_ALPHA_TO_COVERAGE	0x809E	32926	Computes temporary coverage value from fragment alpha.
SAMPLE_COVERAGE	0x80A0	32928	Masks fragments using sample coverage value (<code>sampleCoverage</code>).

3.10 Error Codes

Return values from `getError()` .

Constant Name	Hex Value	Decimal	Condition Triggering Error
<code>NO_ERROR</code>	<code>0</code>	0	No error has occurred since last <code>getError()</code> .
<code>INVALID_ENUM</code>	<code>0x0500</code>	1280	An unacceptable value is specified for an enumerated argument.
<code>INVALID_VALUE</code>	<code>0x0501</code>	1281	A numeric argument is out of range.
<code>INVALID_OPERATION</code>	<code>0x0502</code>	1282	Specified command is not allowed for the current state.
<code>OUT_OF_MEMORY</code>	<code>0x0505</code>	1285	Not enough memory left to execute the command.
<code>INVALID_FRAMEBUFFER_OPERATION</code>	<code>0x0506</code>	1286	Framebuffer object is not complete (<code>checkFramebufferStatus</code>).
<code>CONTEXT_LOST_WEBGL</code>	<code>0x9242</code>	37442	WebGL context has been lost (GPU hang, reset, or device lost).

3.11 Parameter Queries (GetPName)

Global context parameters queried using `getParameter()` .

Constant Name	Hex Value	Decimal	Return Type / Description
<code>LINE_WIDTH</code>	<code>0x0B21</code>	2849	<code>GLfloat</code> - Current rasterized line width.
<code>ALIASED_POINT_SIZE_RANGE</code>	<code>0x846D</code>	33901	<code>Float32Array(2)</code> - Minimum and maximum supported point sizes.
<code>ALIASED_LINE_WIDTH_RANGE</code>	<code>0x846E</code>	33902	<code>Float32Array(2)</code> - Minimum and maximum supported line widths.
<code>CULL_FACE_MODE</code>	<code>0x0B45</code>	2885	<code>GLenum</code> - Polygon cull mode (<code>FRONT</code> , <code>BACK</code> , <code>FRONT_AND_BACK</code>).
<code>FRONT_FACE</code>	<code>0x0B46</code>	2886	<code>GLenum</code> - Front face winding direction (<code>CW</code> or <code>CCW</code>).
<code>DEPTH_RANGE</code>	<code>0x0B70</code>	2928	<code>Float32Array(2)</code> - Near and far depth mapping range.
<code>DEPTH_WRITEMASK</code>	<code>0x0B72</code>	2930	<code>GLboolean</code> - Depth buffer write enablement.
<code>DEPTH_CLEAR_VALUE</code>	<code>0x0B73</code>	2931	<code>GLfloat</code> - Value used to clear depth buffer.
<code>DEPTH_FUNC</code>	<code>0x0B74</code>	2932	<code>GLenum</code> - Current depth comparison function.
<code>VIEWPORT</code>	<code>0x0BA2</code>	2978	<code>Int32Array(4)</code> - Current viewport rectangle [<code>x</code> , <code>y</code> , <code>width</code> , <code>height</code>] .
<code>SCISSOR_BOX</code>	<code>0x0C10</code>	3088	<code>Int32Array(4)</code> - Current scissor rectangle [<code>x</code> , <code>y</code> , <code>width</code> , <code>height</code>] .

COLOR_CLEAR_VALUE	0x0C22	3106	Float32Array(4) - Clear color RGBA values.
COLOR_WRITEMASK	0x0C23	3107	sequence<GLboolean> (4) - Per-channel color write masks.
UNPACK_ALIGNMENT	0x0CF5	3317	GLint - Byte alignment for unpacking pixels from memory (1, 2, 4, 8).
PACK_ALIGNMENT	0x0D05	3333	GLint - Byte alignment for packing pixels into memory.
MAX_TEXTURE_SIZE	0x0D33	3379	GLint - Maximum 2D texture dimensions (width & height).
MAX_VIEWPORT_DIMS	0x0D3A	3386	Int32Array(2) - Maximum supported viewport dimensions.
SUBPIXEL_BITS	0x0D50	3408	GLint - Subpixel resolution bits for rasterization.
RED_BITS	0x0D52	3410	GLint - Bit depth of default buffer red component.
GREEN_BITS	0x0D53	3411	GLint - Bit depth of default buffer green component.
BLUE_BITS	0x0D54	3412	GLint - Bit depth of default buffer blue component.
ALPHA_BITS	0x0D55	3413	GLint - Bit depth of default buffer alpha component.
DEPTH_BITS	0x0D56	3414	GLint - Bit depth of depth buffer.
STENCIL_BITS	0x0D57	3415	GLint - Bit depth of stencil buffer.
POLYGON_OFFSET_UNITS	0x2A00	10752	GLfloat - Value added to polygon depth values.
POLYGON_OFFSET_FACTOR	0x8038	32824	GLfloat - Variable slope factor for polygon depth offset.
TEXTURE_BINDING_2D	0x8069	32873	WebGLTexture? - Currently bound 2D texture on active unit.
SAMPLE_BUFFERS	0x80A8	32936	GLint - Number of multisample buffers (0 or 1).
SAMPLES	0x80A9	32937	GLint - Number of multisample samples per pixel.
SAMPLE_COVERAGE_VALUE	0x80AA	32938	GLfloat - Current sample coverage value.
SAMPLE_COVERAGE_INVERT	0x80AB	32939	GLboolean - Whether sample coverage mask is inverted.

3.12 Hints (HintTarget & HintMode)

Used with `hint(target, mode)` .

Constant Name	Hex Value	Decimal	Category	Description
DONT_CARE	0x1100	4352	HintMode	Implementation decides performance/quality trade-off.
FASTEST	0x1101	4353	HintMode	Prefers highest execution speed over rendering quality.
NICEST	0x1102	4354	HintMode	Prefers highest rendering quality over speed.

GENERATE_MIPMAP_HINT	0x8192	33170	HintTarget	Hint target controlling texture mipmap generation quality.
----------------------	--------	-------	------------	--

3.13 Data Types (DataType)

Fundamental data types for vertex arrays, index lists, and texture formats.

Constant Name	Hex Value	Decimal	Size (bytes)	GL Type / TypedArray Mapping
BYTE	0x1400	5120	1	Signed 8-bit integer (Int8Array)
UNSIGNED_BYTE	0x1401	5121	1	Unsigned 8-bit integer (Uint8Array)
SHORT	0x1402	5122	2	Signed 16-bit integer (Int16Array)
UNSIGNED_SHORT	0x1403	5123	2	Unsigned 16-bit integer (Uint16Array)
INT	0x1404	5124	4	Signed 32-bit integer (Int32Array)
UNSIGNED_INT	0x1405	5125	4	Unsigned 32-bit integer (Uint32Array)
FLOAT	0x1406	5126	4	32-bit IEEE 754 Floating point (Float32Array)

3.14 Pixel Formats & Pixel Types

PixelFormat

Internal and transfer color formats for textures and readPixels() .

Constant Name	Hex Value	Decimal	Channels	Description
DEPTH_COMPONENT	0x1902	6402	1	Single depth component.
ALPHA	0x1906	6406	1	Alpha transparency channel only.
RGB	0x1907	6407	3	Red, Green, Blue color channels.
RGBA	0x1908	6408	4	Red, Green, Blue, Alpha channels.
LUMINANCE	0x1909	6409	1	Grayscale luminance (duplicated across RGB in shaders).
LUMINANCE_ALPHA	0x190A	6410	2	Grayscale luminance + Alpha.

PixelFormat

Packed pixel component encodings for textures.

Constant Name	Hex Value	Decimal	Bits	Encoding Breakdown
UNSIGNED_BYTE	0x1401	5121	8 / chan	8 bits per channel (used with RGBA, RGB, etc.).
UNSIGNED_SHORT_4_4_4_4	0x8033	32819	16	Packed RGBA (4 bits R, 4 bits G, 4 bits B, 4 bits A).
UNSIGNED_SHORT_5_5_5_1	0x8034	32820	16	Packed RGBA (5 bits R, 5 bits G, 5 bits B, 1 bit A).
UNSIGNED_SHORT_5_6_5	0x8363	33635	16	Packed RGB (5 bits R, 6 bits G, 5 bits B).

3.15 Shaders, Programs & Precision Types

Shaders & Program Constants

Constant Name	Hex Value	Decimal	Classification	Description
FRAGMENT_SHADER	0x8B30	35632	Shader Type	Compiles fragment (pixel) processing stage.
VERTEX_SHADER	0x8B31	35633	Shader Type	Compiles vertex transformation stage.
MAX_VERTEX_ATTRIBS	0x8869	34921	Limit Query	Maximum number of 4-component vertex attributes.
MAX_VERTEX_UNIFORM_VECTORS	0x8DFB	36347	Limit Query	Maximum 4-element floating-point uniform vectors in vertex shader.
MAX_VARYING_VECTORS	0x8DFC	36348	Limit Query	Maximum 4-element floating-point varying vectors.
MAX_COMBINED_TEXTURE_IMAGE_UNITS	0x8B4D	35661	Limit Query	Total texture units accessible across all shader stages.
MAX_VERTEX_TEXTURE_IMAGE_UNITS	0x8B4C	35660	Limit Query	Maximum texture units accessible within vertex shaders.
MAX_TEXTURE_IMAGE_UNITS	0x8872	34930	Limit Query	Maximum texture units accessible within fragment shaders.
MAX_FRAGMENT_UNIFORM_VECTORS	0x8DFD	36349	Limit Query	Maximum 4-element floating-point uniform vectors in fragment shader.
SHADER_TYPE	0x8B4F	35663	Query	Parameter query returning shader type (VERTEX_ or FRAGMENT_SHADER).
DELETE_STATUS	0x8B80	35712	Query	Returns GL_TRUE if shader or program is marked for deletion.
COMPILE_STATUS	0x8B81	35713	Query	Returns GL_TRUE if previous shader compilation succeeded.
LINK_STATUS	0x8B82	35714	Query	Returns GL_TRUE if previous program linking succeeded.
VALIDATE_STATUS	0x8B83	35715	Query	Returns GL_TRUE if program validation succeeded.
ATTACHED_SHADERS	0x8B85	35717	Query	Number of attached shaders to a program.
ACTIVE_UNIFORMS	0x8B86	35718	Query	Number of active uniforms in a program.

ACTIVE_ATTRIBUTES	0x8B89	35721	Query	Number of active attributes in a program.
SHADING_LANGUAGE_VERSION	0x8B8C	35724	String Query	Returns supported shading language version string.
CURRENT_PROGRAM	0x8B8D	35725	Query	Returns currently active WebGLProgram .

Shader Precision-Specified Types

Parameters passed to `getShaderPrecisionFormat(shaderType, precisionType)` .

Constant Name	Hex Value	Decimal	Classification	Description
LOW_FLOAT	0x8DF0	36336	Float Precision	Low precision floating point qualifier (<code>lowp_float</code>).
MEDIUM_FLOAT	0x8DF1	36337	Float Precision	Medium precision floating point qualifier (<code>mediump_float</code>).
HIGH_FLOAT	0x8DF2	36338	Float Precision	High precision floating point qualifier (<code>highp_float</code>).
LOW_INT	0x8DF3	36339	Int Precision	Low precision integer qualifier (<code>lowp_int</code>).
MEDIUM_INT	0x8DF4	36340	Int Precision	Medium precision integer qualifier (<code>mediump_int</code>).
HIGH_INT	0x8DF5	36341	Int Precision	High precision integer qualifier (<code>highp_int</code>).

3.16 Uniform Types

Enums returned in `WebGLActiveInfo.type` describing active uniform types in shaders.

Constant Name	Hex Value	Decimal	GLSL Representation
FLOAT_VEC2	0x8B50	35664	<code>vec2</code> (two-element float vector)
FLOAT_VEC3	0x8B51	35665	<code>vec3</code> (three-element float vector)
FLOAT_VEC4	0x8B52	35666	<code>vec4</code> (four-element float vector)
INT_VEC2	0x8B53	35667	<code>ivec2</code> (two-element integer vector)
INT_VEC3	0x8B54	35668	<code>ivec3</code> (three-element integer vector)
INT_VEC4	0x8B55	35669	<code>ivec4</code> (four-element integer vector)
BOOL	0x8B56	35670	<code>bool</code> (boolean)
BOOL_VEC2	0x8B57	35671	<code>bvec2</code> (two-element boolean vector)
BOOL_VEC3	0x8B58	35672	<code>bvec3</code> (three-element boolean vector)
BOOL_VEC4	0x8B59	35673	<code>bvec4</code> (four-element boolean vector)

<code>FLOAT_MAT2</code>	<code>0x8B5A</code>	35674	<code>mat2</code> (2x2 float matrix)
<code>FLOAT_MAT3</code>	<code>0x8B5B</code>	35675	<code>mat3</code> (3x3 float matrix)
<code>FLOAT_MAT4</code>	<code>0x8B5C</code>	35676	<code>mat4</code> (4x4 float matrix)
<code>SAMPLER_2D</code>	<code>0x8B5E</code>	35678	<code>sampler2D</code> (2D texture sampler)
<code>SAMPLER_CUBE</code>	<code>0x8B60</code>	35680	<code>samplerCube</code> (Cube map texture sampler)

3.17 Vertex Arrays & Attributes

Parameters queried with `getVertexAttrib()` .

Constant Name	Hex Value	Decimal	Description
<code>VERTEX_ATTRIB_ARRAY_ENABLED</code>	<code>0x8622</code>	34338	<code>GLboolean</code> - Whether attribute array is enabled at this index.
<code>VERTEX_ATTRIB_ARRAY_SIZE</code>	<code>0x8623</code>	34339	<code>GLint</code> - Component size of vertex attribute (1, 2, 3, 4).
<code>VERTEX_ATTRIB_ARRAY_STRIDE</code>	<code>0x8624</code>	34340	<code>GLsizei</code> - Byte offset between consecutive vertex attributes.
<code>VERTEX_ATTRIB_ARRAY_TYPE</code>	<code>0x8625</code>	34341	<code>GLenum</code> - Data type (<code>FLOAT</code> , <code>BYTE</code> , <code>UNSIGNED_SHORT</code> , etc.).
<code>VERTEX_ATTRIB_ARRAY_NORMALIZED</code>	<code>0x886A</code>	34922	<code>GLboolean</code> - Whether fixed-point values are normalized to <code>[-1, 1]</code> or <code>[0, 1]</code> .
<code>VERTEX_ATTRIB_ARRAY_POINTER</code>	<code>0x8645</code>	34373	<code>GLintptr</code> - Pointer / offset into bound buffer (<code>getVertexAttribOffset</code>).
<code>VERTEX_ATTRIB_ARRAY_BUFFER_BINDING</code>	<code>0x889F</code>	34975	<code>WebGLBuffer?</code> - Buffer bound to this vertex attribute.

3.18 Textures (Targets, Units, Filters, Wrap Modes & Parameters)

Texture Targets

Constant Name	Hex Value	Decimal	Description
<code>TEXTURE_2D</code>	<code>0x0DE1</code>	3553	Standard 2D texture target.
<code>TEXTURE</code>	<code>0x1702</code>	5890	Texture target query parameter.
<code>TEXTURE_CUBE_MAP</code>	<code>0x8513</code>	34067	Cube map texture target containing 6 square faces.
<code>TEXTURE_BINDING_CUBE_MAP</code>	<code>0x8514</code>	34068	Currently bound cube map texture query.
<code>TEXTURE_CUBE_MAP_POSITIVE_X</code>	<code>0x8515</code>	34069	Cube map face: Positive X (+X).
<code>TEXTURE_CUBE_MAP_NEGATIVE_X</code>	<code>0x8516</code>	34070	Cube map face: Negative X (-X).

TEXTURE_CUBE_MAP_POSITIVE_Y	0x8517	34071	Cube map face: Positive Y (+Y).
TEXTURE_CUBE_MAP_NEGATIVE_Y	0x8518	34072	Cube map face: Negative Y (-Y).
TEXTURE_CUBE_MAP_POSITIVE_Z	0x8519	34073	Cube map face: Positive Z (+Z).
TEXTURE_CUBE_MAP_NEGATIVE_Z	0x851A	34074	Cube map face: Negative Z (-Z).
MAX_CUBE_MAP_TEXTURE_SIZE	0x851C	34076	Maximum cube map dimension (width/height per face).
COMPRESSED_TEXTURE_FORMATS	0x86A3	34467	List of supported compressed texture formats.

Texture Parameter Names & Wrap Modes

Constant Name	Hex Value	Decimal	Classification	Description
TEXTURE_MAG_FILTER	0x2800	10240	Parameter Name	Magnification filter function.
TEXTURE_MIN_FILTER	0x2801	10241	Parameter Name	Minification filter function.
TEXTURE_WRAP_S	0x2802	10242	Parameter Name	Wrap parameter for horizontal coordinate S (U).
TEXTURE_WRAP_T	0x2803	10243	Parameter Name	Wrap parameter for vertical coordinate T (V).
REPEAT	0x2901	10497	Wrap Mode	Repeats the texture image tiles infinitely.
CLAMP_TO_EDGE	0x812F	33071	Wrap Mode	Clamps coordinates to [0.0, 1.0] , clamping edges.
MIRRORED_REPEAT	0x8370	33648	Wrap Mode	Repeats texture image alternating mirrored flips.

Texture Filters

Constant Name	Hex Value	Decimal	Min/Mag Applicability	Sampling Operation
NEAREST	0x2600	9728	Both Min & Mag	Nearest neighbor point sampling (pixelated).
LINEAR	0x2601	9729	Both Min & Mag	Bilinear filtering (weighted 2x2 average).
NEAREST_MIPMAP_NEAREST	0x2700	9984	Min Only	Nearest pixel from nearest mipmap level.
LINEAR_MIPMAP_NEAREST	0x2701	9985	Min Only	Bilinear interpolation within nearest mipmap level.
NEAREST_MIPMAP_LINEAR	0x2702	9986	Min Only	Nearest pixel linearly blended across 2 closest mipmaps.
LINEAR_MIPMAP_LINEAR	0x2703	9987	Min Only	Trilinear filtering (bilinear sample from 2 mipmaps blended).

Texture Units

Active texture units selected via `activeTexture(TEXTURE0 + n)` .

Constant Name	Hex Value	Decimal	Constant Name	Hex Value	Decimal
TEXTURE0	0x84C0	33984	TEXTURE16	0x84D0	34000
TEXTURE1	0x84C1	33985	TEXTURE17	0x84D1	34001
TEXTURE2	0x84C2	33986	TEXTURE18	0x84D2	34002
TEXTURE3	0x84C3	33987	TEXTURE19	0x84D3	34003
TEXTURE4	0x84C4	33988	TEXTURE20	0x84D4	34004
TEXTURE5	0x84C5	33989	TEXTURE21	0x84D5	34005
TEXTURE6	0x84C6	33990	TEXTURE22	0x84D6	34006
TEXTURE7	0x84C7	33991	TEXTURE23	0x84D7	34007
TEXTURE8	0x84C8	33992	TEXTURE24	0x84D8	34008
TEXTURE9	0x84C9	33993	TEXTURE25	0x84D9	34009
TEXTURE10	0x84CA	33994	TEXTURE26	0x84DA	34010
TEXTURE11	0x84CB	33995	TEXTURE27	0x84DB	34011
TEXTURE12	0x84CC	33996	TEXTURE28	0x84DC	34012
TEXTURE13	0x84CD	33997	TEXTURE29	0x84DD	34013
TEXTURE14	0x84CE	33998	TEXTURE30	0x84DE	34014
TEXTURE15	0x84CF	33999	TEXTURE31	0x84DF	34015
ACTIVE_TEXTURE	0x84E0	34016	—	—	Current active texture unit query.

3.19 Framebuffers & Renderbuffers

Core Types, Bindings & Limits

Constant Name	Hex Value	Decimal	Classification	Description
FRAMEBUFFER	0x8D40	36160	Target	Framebuffer binding target.
RENDERBUFFER	0x8D41	36161	Target	Renderbuffer binding target.
FRAMEBUFFER_BINDING	0x8CA6	36006	Query	Currently bound <code>WebGLFramebuffer</code> .
RENDERBUFFER_BINDING	0x8CA7	36007	Query	Currently bound <code>WebGLRenderbuffer</code> .
MAX_RENDERBUFFER_SIZE	0x84E8	34024	Query Limit	Maximum width & height of renderbuffers.
NONE	0	0	Value	Unbound / empty attachment or parameter state.

Internal Storage Formats

Constant Name	Hex Value	Decimal	Channel Details	Use Case
RGBA4	0x8056	32854	4 bits R, G, B, A each	Color renderbuffer
RGB5_A1	0x8057	32855	5 bits R, G, B, 1 bit A	Color renderbuffer
RGBA8	0x8058	32856	8 bits R, G, B, A each	Color renderbuffer
RGB565	0x8D62	36194	5 bits R, 6 bits G, 5 bits B	Color renderbuffer
DEPTH_COMPONENT16	0x81A5	33189	16-bit depth component	Depth renderbuffer
STENCIL_INDEX8	0x8D48	36168	8-bit stencil index	Stencil renderbuffer
DEPTH_STENCIL	0x84F9	34041	Packed depth & stencil format	Depth-stencil attachments

Renderbuffer Parameter Queries

Parameters queried via `getRenderbufferParameter(RENDERBUFFER, pname)` .

Constant Name	Hex Value	Decimal	Description
RENDERBUFFER_WIDTH	0x8D42	36162	Renderbuffer image width in pixels.
RENDERBUFFER_HEIGHT	0x8D43	36163	Renderbuffer image height in pixels.
RENDERBUFFER_INTERNAL_FORMAT	0x8D44	36164	Sized internal format (<code>RGBA4</code> , <code>DEPTH_COMPONENT16</code> , etc.).
RENDERBUFFER_RED_SIZE	0x8D50	36176	Actual resolution of red component in bits.
RENDERBUFFER_GREEN_SIZE	0x8D51	36177	Actual resolution of green component in bits.
RENDERBUFFER_BLUE_SIZE	0x8D52	36178	Actual resolution of blue component in bits.
RENDERBUFFER_ALPHA_SIZE	0x8D53	36179	Actual resolution of alpha component in bits.
RENDERBUFFER_DEPTH_SIZE	0x8D54	36180	Actual resolution of depth component in bits.
RENDERBUFFER_STENCIL_SIZE	0x8D55	36181	Actual resolution of stencil component in bits.

Framebuffer Attachments & Attachment Queries

Used in `framebufferTexture2D` , `framebufferRenderbuffer` , and `getFramebufferAttachmentParameter` .

Constant Name	Hex Value	Decimal	Classification	Description
COLOR_ATTACHMENT0	0x8CE0	36064	Attachment	Color buffer attachment point 0.
DEPTH_ATTACHMENT	0x8D00	36096	Attachment	Depth buffer attachment point.

STENCIL_ATTACHMENT	0x8D20	36128	Attachment	Stencil buffer attachment point.
DEPTH_STENCIL_ATTACHMENT	0x821A	33306	Attachment	Joint depth and stencil attachment.
FRAMEBUFFER_ATTACHMENT_OBJECT_TYPE	0x8CD0	36048	Parameter Name	Object type (RENDERBUFFER , TEXTURE , NONE).
FRAMEBUFFER_ATTACHMENT_OBJECT_NAME	0x8CD1	36049	Parameter Name	Target object (WebGLRenderbuffer or WebGLTexture).
FRAMEBUFFER_ATTACHMENT_TEXTURE_LEVEL	0x8CD2	36050	Parameter Name	Mipmap level attached.
FRAMEBUFFER_ATTACHMENT_TEXTURE_CUBE_MAP_FACE	0x8CD3	36051	Parameter Name	Attached cube map face.

Framebuffer Completeness Status

Returned from `checkFramebufferStatus(FRAMEBUFFER)` .

Constant Name	Hex Value	Decimal	Meaning / Reason
FRAMEBUFFER_COMPLETE	0x8CD5	36053	Framebuffer is complete and ready for rendering.
FRAMEBUFFER_INCOMPLETE_ATTACHMENT	0x8CD6	36054	An attachment point is uninitialized or has invalid format.
FRAMEBUFFER_INCOMPLETE_MISSING_ATTACHMENT	0x8CD7	36055	No images are attached to the framebuffer.
FRAMEBUFFER_INCOMPLETE_DIMENSIONS	0x8CD9	36057	Attached images do not have matching width and height.
FRAMEBUFFER_UNSUPPORTED	0x8CDD	36061	Combination of internal formats is unsupported by hardware.

3.20 Read Format Queries

Implementation-dependent pixel readback queries passed to `getParameter()` .

Constant Name	Hex Value	Decimal	Description
IMPLEMENTATION_COLOR_READ_TYPE	0x8B9A	35738	Returns optimal <code>type</code> argument for <code>readPixels()</code> .
IMPLEMENTATION_COLOR_READ_FORMAT	0x8B9B	35739	Returns optimal <code>format</code> argument for <code>readPixels()</code> .

3.21 Implementation Strings (StringName)

Passed to `getParameter(name)` to query driver metadata.

Constant Name	Hex Value	Decimal	Returned Content
VENDOR	0x1F00	7936	Company/vendor responsible for WebGL implementation.
RENDERER	0x1F01	7937	Hardware renderer name or virtual GPU identity.
VERSION	0x1F02	7938	WebGL specification and driver release version.

3.22 WebGL-Specific Constants

Constants unique to the HTML WebGL browser specification (used in `pixelStorei` and loss handlers).

Constant Name	Hex Value	Decimal	Description
UNPACK_FLIP_Y_WEBGL	0x9240	37440	<code>pixelStorei</code> boolean parameter: flips uploaded images along vertical Y axis to match WebGL coordinates.
UNPACK_PREMULTIPLY_ALPHA_WEBGL	0x9241	37441	<code>pixelStorei</code> boolean parameter: multiplies RGB channels by Alpha during image upload.
CONTEXT_LOST_WEBGL	0x9242	37442	Error code returned by <code>getError()</code> when context loss has occurred.
UNPACK_COLORSPACE_CONVERSION_WEBGL	0x9243	37443	<code>pixelStorei</code> parameter controlling image color space conversion during texture upload.
BROWSER_DEFAULT_WEBGL	0x9244	37444	Setting for <code>UNPACK_COLORSPACE_CONVERSION_WEBGL</code> specifying default browser color conversion behavior.

4. Exhaustive Function Reference

4.1 Context Lifecycle, Capabilities & Extensions

`isContextLost()`

Spec Declaration:

```
[WebGLHandlesContextLoss] boolean isContextLost();
```

- Description:** Returns `true` if the underlying GPU context was lost (e.g. driver crash, out-of-memory, or tab backgrounding), and `false` otherwise.
- Special Annotation:** Decorated with `[WebGLHandlesContextLoss]` allowing execution even after context loss.

`getContextAttributes()`

Spec Declaration:

```
[WebGLHandlesContextLoss] WebGLContextAttributes? getContextAttributes();
```

- **Description:** Returns a dictionary describing the actual context attributes used to initialize the context (e.g., `alpha`, `depth`, `stencil`, `antialias`, `premultipliedAlpha`, `preserveDrawingBuffer`). Returns `null` if context is lost.

getSupportedExtensions()

Spec Declaration:

```
sequence<DOMString>? getSupportedExtensions();
```

- **Description:** Returns a list of supported extension strings (e.g., `"OES_texture_float"`, `"ANGLE_instanced_arrays"`), or `null` if lost.

getExtension()

Spec Declaration:

```
object? getExtension(DOMString name);
```

- **Description:** Activates and returns the requested extension object, or `null` if unsupported.

drawingBufferStorage()

Spec Declaration:

```
undefined drawingBufferStorage(GLenum sizedFormat, unsigned long width, unsigned long height);
```

- **Description:** Configures custom internal storage size and format for the default drawing buffer.

enable()

Spec Declaration:

```
undefined enable(GLenum cap);
```

- **Description:** Enables a specific fixed-function WebGL capability (`BLEND`, `CULL_FACE`, `DEPTH_TEST`, `DITHER`, `POLYGON_OFFSET_FILL`, `SAMPLE_ALPHA_TO_COVERAGE`, `SAMPLE_COVERAGE`, `SCISSOR_TEST`, `STENCIL_TEST`).

disable()

Spec Declaration:

```
undefined disable(GLenum cap);
```

- **Description:** Disables the specified fixed-function capability.

isEnabled()

Spec Declaration:

```
[WebGLHandlesContextLoss] GLboolean isEnabled(GLenum cap);
```

- **Description:** Queries whether a specified capability is currently enabled.

getError()

Spec Declaration:

```
[WebGLHandlesContextLoss] GLenum getError();
```

- **Description:** Returns the earliest recorded error flag (`NO_ERROR` , `INVALID_ENUM` , `INVALID_VALUE` , `INVALID_OPERATION` , `OUT_OF_MEMORY` , `INVALID_FRAMEBUFFER_OPERATION` , `CONTEXT_LOST_WEBGL`) and resets error state to `NO_ERROR` .

getParameter()

Spec Declaration:

```
any getParameter(GLenum pname);
```

- **Description:** Queries the current value of a global state parameter or hardware capability limit (see table in Section 3.11).

hint()

Spec Declaration:

```
undefined hint(GLenum target, GLenum mode);
```

- **Parameters:** `target` (`GENERATE_MIPMAP_HINT`), `mode` (`DONT_CARE` , `FASTEST` , `NICEST`).
- **Description:** Specifies implementation-dependent rendering quality hints.

4.2 Frame Clearing & Viewport / Scissor Rectangles

viewport()

Spec Declaration:

```
undefined viewport(GLint x, GLint y, GLsizei width, GLsizei height);
```

- **Description:** Sets the viewport affine transformation that maps normalized device coordinates (NDC [-1, 1]) to window pixel coordinates.

scissor()

Spec Declaration:

```
undefined scissor(GLint x, GLint y, GLsizei width, GLsizei height);
```

- **Description:** Defines the scissor box rectangle in window coordinates. When `SCISSOR_TEST` is enabled, only fragments inside this box are updated.

`clear()`

Spec Declaration:

```
undefined clear(GLbitfield mask);
```

- **Parameters:** `mask` - Bitwise OR of `COLOR_BUFFER_BIT`, `DEPTH_BUFFER_BIT`, and `STENCIL_BUFFER_BIT`.
- **Description:** Clears the specified active render buffers to their currently configured clear values.

`clearColor()`

Spec Declaration:

```
undefined clearColor(GLclampf red, GLclampf green, GLclampf blue, GLclampf alpha);
```

- **Description:** Sets the RGBA color components used when clearing the color buffer (clamped to `[0.0, 1.0]`).

`clearDepth()`

Spec Declaration:

```
undefined clearDepth(GLclampf depth);
```

- **Description:** Sets the depth value used when clearing the depth buffer (clamped to `[0.0, 1.0]`, defaults to `1.0`).

`clearStencil()`

Spec Declaration:

```
undefined clearStencil(GLint s);
```

- **Description:** Sets the integer index value used when clearing the stencil buffer.

4.3 Rasterizer State & Depth / Stencil / Color Masking

`colorMask()`

Spec Declaration:

```
undefined colorMask(GLboolean red, GLboolean green, GLboolean blue, GLboolean alpha);
```

- **Description:** Enables or disables writing of individual color components into the color buffer.

`depthMask()`

Spec Declaration:

```
undefined depthMask(GLboolean flag);
```

- **Description:** Enables (`true`) or disables (`false`) writing into the depth buffer.

`depthFunc()`

Spec Declaration:

```
undefined depthFunc(GLenum func);
```

- **Description:** Sets the comparison function used in the depth-buffer test (`NEVER` , `LESS` , `EQUAL` , `LEQUAL` , `GREATER` , `NOTEQUAL` , `GEQUAL` , `ALWAYS`). Defaults to `LESS` .

`depthRange()`

Spec Declaration:

```
undefined depthRange(GLclampf zNear, GLclampf zFar);
```

- **Description:** Specifies linear mapping of normalized device coordinate depth values [-1, 1] to window depth values [`zNear` , `zFar`] . Defaults to [0.0, 1.0] .

`cullFace()`

Spec Declaration:

```
undefined cullFace(GLenum mode);
```

- **Description:** Sets whether front-facing, back-facing, or both facets are culled (`FRONT` , `BACK` , `FRONT_AND_BACK`).

`frontFace()`

Spec Declaration:

```
undefined frontFace(GLenum mode);
```

- **Description:** Defines whether clockwise (`CW`) or counter-clockwise (`CCW`) polygon winding indicates front-facing polygons. Defaults to `CCW` .

`lineWidth()`

Spec Declaration:

```
undefined lineWidth(GLfloat width);
```

- **Description:** Sets the rasterized width of lines drawn with primitive topologies `LINES` , `LINE_STRIP` , or `LINE_LOOP` .

`polygonOffset()`

Spec Declaration:

```
undefined polygonOffset(GLfloat factor, GLfloat units);
```

- **Description:** Sets scale factors for computing polygon depth bias to eliminate z-fighting artifacts on coplanar geometry.

`sampleCoverage()`

Spec Declaration:

```
undefined sampleCoverage(GLclampf value, GLboolean invert);
```

- **Description:** Sets multisample coverage parameters when `SAMPLE_COVERAGE` is enabled.

`stencilFunc()`

Spec Declaration:

```
undefined stencilFunc(GLenum func, GLint ref, GLuint mask);
```

- **Description:** Sets front-facing and back-facing stencil test function, reference value, and comparison mask.

`stencilFuncSeparate()`

Spec Declaration:

```
undefined stencilFuncSeparate(GLenum face, GLenum func, GLint ref, GLuint mask);
```

- **Description:** Sets stencil test parameters separately for specified face (`FRONT` , `BACK` , or `FRONT_AND_BACK`).

`stencilMask()`

Spec Declaration:

```
undefined stencilMask(GLuint mask);
```

- **Description:** Sets a bitmask enabling and disabling writing of individual bits in the stencil buffer for both faces.

`stencilMaskSeparate()`

Spec Declaration:

```
undefined stencilMaskSeparate(GLenum face, GLuint mask);
```

- **Description:** Sets the stencil write mask separately for specified face (`FRONT` , `BACK` , or `FRONT_AND_BACK`).

`stencilOp()`

Spec Declaration:

```
undefined stencilOp(GLenum fail, GLenum zfail, GLenum zpass);
```

- **Description:** Sets front and back stencil buffer actions:
 - `fail` : Action when stencil test fails.
 - `zfail` : Action when stencil test passes but depth test fails.

- `zpass` : Action when both stencil and depth tests pass.

`stencilOpSeparate()`

Spec Declaration:

```
undefined stencilOpSeparate(GLenum face, GLenum fail, GLenum zfail, GLenum zpass);
```

- **Description:** Sets stencil actions separately for specified face (`FRONT` , `BACK` , or `FRONT_AND_BACK`).

4.4 Blending Configuration

`blendColor()`

Spec Declaration:

```
undefined blendColor(GLclampf red, GLclampf green, GLclampf blue, GLclampf alpha);
```

- **Description:** Sets the constant blend color used in `CONSTANT_COLOR` and `ONE_MINUS_CONSTANT_COLOR` blending equations.

`blendEquation()`

Spec Declaration:

```
undefined blendEquation(GLenum mode);
```

- **Description:** Sets both RGB and Alpha blend equations (`FUNC_ADD` , `FUNC_SUBTRACT` , `FUNC_REVERSE_SUBTRACT`).

`blendEquationSeparate()`

Spec Declaration:

```
undefined blendEquationSeparate(GLenum modeRGB, GLenum modeAlpha);
```

- **Description:** Sets the RGB blend equation and Alpha blend equation independently.

`blendFunc()`

Spec Declaration:

```
undefined blendFunc(GLenum sfactor, GLenum dfactor);
```

- **Description:** Specifies pixel arithmetic for both RGB and Alpha channels for source (`sfactor`) and destination (`dfactor`).

`blendFuncSeparate()`

Spec Declaration:

```
undefined blendFuncSeparate(GLenum srcRGB, GLenum dstRGB, GLenum srcAlpha, GLenum dstAlpha);
```

- **Description:** Sets pixel arithmetic blending factors for RGB and Alpha channels independently.
-

4.5 Buffer Management & Data Upload

`createBuffer()`

Spec Declaration:

```
WebGLBuffer createBuffer();
```

- **Description:** Creates and returns a new uninitialized `WebGLBuffer` object.

`bindBuffer()`

Spec Declaration:

```
undefined bindBuffer(GGLenum target, WebGLBuffer? buffer);
```

- **Parameters:** `target` (`ARRAY_BUFFER` or `ELEMENT_ARRAY_BUFFER`), `buffer` (or `null` to unbind).
- **Description:** Binds a `WebGLBuffer` to the designated buffer target.

`bufferData()`

Spec Declaration:

```
undefined bufferData(GGLenum target, GLsizeiptr size, GGLenum usage);
```

- **Description:** Allocates a new data store of `size` bytes for the buffer bound to `target` with usage hint (`STATIC_DRAW`, `DYNAMIC_DRAW`, `STREAM_DRAW`).

`bufferData()`

Spec Declaration:

```
undefined bufferData(GGLenum target, AllowSharedBufferSource? data, GGLenum usage);
```

- **Description:** Initializes and creates a buffer data store populated with data from an `ArrayBuffer` or `ArrayBufferView` (including shared buffers).

`bufferSubData()`

Spec Declaration:

```
undefined bufferSubData(GGLenum target, GLintptr offset, AllowSharedBufferSource data);
```

- **Description:** Updates a contiguous subregion of the currently bound buffer starting at byte `offset` using values from `data`.

`deleteBuffer()`

Spec Declaration:

```
undefined deleteBuffer(WebGLBuffer? buffer);
```

- **Description:** Deletes the specified `WebGLBuffer`, freeing its allocated GPU resources.

`isBuffer()`

Spec Declaration:

```
[WebGLHandlesContextLoss] GLboolean isBuffer(WebGLBuffer? buffer);
```

- **Description:** Returns `true` if the passed object is an existing, valid `WebGLBuffer`.

`getBufferParameter()`

Spec Declaration:

```
any getBufferParameter(GLenum target, GLenum pname);
```

- **Description:** Returns the value of a buffer parameter (`BUFFER_SIZE`, `BUFFER_USAGE`) for the bound buffer.

4.6 Shaders & Programs Pipeline

`createShader()`

Spec Declaration:

```
WebGLShader? createShader(GLenum type);
```

- **Parameters:** `type` (`VERTEX_SHADER` or `FRAGMENT_SHADER`).
- **Description:** Creates and returns an empty `WebGLShader` object, or `null` on failure.

`shaderSource()`

Spec Declaration:

```
undefined shaderSource(WebGLShader shader, DOMString source);
```

- **Description:** Sets the GLSL source code string for the specified shader.

`compileShader()`

Spec Declaration:

```
undefined compileShader(WebGLShader shader);
```

- **Description:** Compiles the GLSL source string loaded into `shader`.

`getShaderParameter()`

Spec Declaration:

```
any getShaderParameter(WebGLShader shader, GLenum pname);
```

- **Parameters:** `pname` (`DELETE_STATUS` , `COMPILE_STATUS` , `SHADER_TYPE`).
- **Description:** Returns compilation status, deletion status, or shader type.

`getShaderInfoLog()`

Spec Declaration:

```
DOMString? getShaderInfoLog(WebGLShader shader);
```

- **Description:** Returns compiler diagnostics, warnings, and compilation error logs for `shader` .

`getShaderSource()`

Spec Declaration:

```
DOMString? getShaderSource(WebGLShader shader);
```

- **Description:** Retrieves the GLSL source code string stored in `shader` .

`getShaderPrecisionFormat()`

Spec Declaration:

```
WebGLShaderPrecisionFormat? getShaderPrecisionFormat(GLenum shadertype, GLenum precisiontype);
```

- **Description:** Returns a `WebGLShaderPrecisionFormat` describing the range and precision (in bits) for the given shader type and precision qualifier (e.g. `LOW_FLOAT` , `HIGH_FLOAT` , `HIGH_INT`).

`deleteShader()`

Spec Declaration:

```
undefined deleteShader(WebGLShader? shader);
```

- **Description:** Marks the shader for deletion. The object is freed when detached from all programs.

`isShader()`

Spec Declaration:

```
[WebGLHandlesContextLoss] GLboolean isShader(WebGLShader? shader);
```

- **Description:** Returns `true` if the argument is a valid `WebGLShader` object.

`createProgram()`

Spec Declaration:

```
WebGLProgram createProgram();
```

- **Description:** Creates and returns a new uninitialized `WebGLProgram` object.

`attachShader()`

Spec Declaration:

```
void attachShader(WebGLProgram program, WebGLShader shader);
```

- **Description:** Attaches a compiled `WebGLShader` to a `WebGLProgram`.

`detachShader()`

Spec Declaration:

```
void detachShader(WebGLProgram program, WebGLShader shader);
```

- **Description:** Detaches an attached `WebGLShader` from a `WebGLProgram`.

`linkProgram()`

Spec Declaration:

```
void linkProgram(WebGLProgram program);
```

- **Description:** Links the vertex and fragment shaders attached to `program` into an executable GPU pipeline.

`useProgram()`

Spec Declaration:

```
void useProgram(WebGLProgram? program);
```

- **Description:** Installs the specified `WebGLProgram` as part of current rendering state (or unbinds if `null`).

`validateProgram()`

Spec Declaration:

```
void validateProgram(WebGLProgram program);
```

- **Description:** Validates `program` against the current WebGL state machine to test if it can execute under current bindings.

`getProgramParameter()`

Spec Declaration:

```
any getProgramParameter(WebGLProgram program, GLenum pname);
```

- **Parameters:** `pname` (`DELETE_STATUS`, `LINK_STATUS`, `VALIDATE_STATUS`, `ATTACHED_SHADERS`, `ACTIVE_ATTRIBUTES`, `ACTIVE_UNIFORMS`).

- **Description:** Returns linking, validation, and count metrics for `program`.

`getProgramInfoLog()`

Spec Declaration:

```
DOMString? getProgramInfoLog(WebGLProgram program);
```

- **Description:** Returns linker diagnostics, warnings, and error messages for `program`.

`getAttachedShaders()`

Spec Declaration:

```
sequence<WebGLShader>? getAttachedShaders(WebGLProgram program);
```

- **Description:** Returns a sequence of all `WebGLShader` objects attached to `program`.

`deleteProgram()`

Spec Declaration:

```
undefined deleteProgram(WebGLProgram? program);
```

- **Description:** Marks `program` for deletion; freed once no longer in use by the active context.

`isProgram()`

Spec Declaration:

```
[WebGLHandlesContextLoss] GLboolean isProgram(WebGLProgram? program);
```

- **Description:** Returns `true` if the object is a valid `WebGLProgram`.

4.7 Vertex Attributes & Pointers

`bindAttribLocation()`

Spec Declaration:

```
undefined bindAttribLocation(WebGLProgram program, GLuint index, DOMString name);
```

- **Description:** Binds a generic vertex attribute index to an attribute variable `name` prior to program linking.

`getAttribLocation()`

Spec Declaration:

```
[WebGLHandlesContextLoss] GLint getAttribLocation(WebGLProgram program, DOMString name);
```

- **Description:** Returns the integer location index assigned to the attribute variable `name` in `program`, or `-1` if not found.

`getActiveAttrib()`

Spec Declaration:

```
WebGLActiveInfo? getActiveAttrib(WebGLProgram program, GLuint index);
```

- **Description:** Returns a `WebGLActiveInfo` object (with `name`, `size`, `type`) for active attribute at `index`.

`enableVertexAttribArray()`

Spec Declaration:

```
undefined enableVertexAttribArray(GLuint index);
```

- **Description:** Enables the generic vertex attribute array at `index` to stream data from bound `ARRAY_BUFFER`.

`disableVertexAttribArray()`

Spec Declaration:

```
undefined disableVertexAttribArray(GLuint index);
```

- **Description:** Disables the vertex attribute array at `index`, falling back to constant values set by `vertexAttrib[1..4]f`.

`vertexAttribPointer()`

Spec Declaration:

```
undefined vertexAttribPointer(GLuint index, GLint size, GLenum type, GLboolean normalized, GLsizei stride, GLintptr offset);
```

- **Parameters:**
 - `index`: Vertex attribute slot index.
 - `size`: Number of components per vertex (1, 2, 3, or 4).
 - `type`: Data type (`BYTE`, `UNSIGNED_BYTE`, `SHORT`, `UNSIGNED_SHORT`, `FLOAT`).
 - `normalized`: If `true`, integer values are mapped to `[-1, 1]` or `[0, 1]`.
 - `stride`: Byte offset between consecutive vertex attributes (0 = tightly packed).
 - `offset`: Byte offset into the currently bound `ARRAY_BUFFER` data store.
- **Description:** Configures how vertex shader attributes read layout and binary structure from the currently bound `ARRAY_BUFFER`.

`getVertexAttrib()`

Spec Declaration:

```
any glVertexAttrib(GLuint index, GLenum pname);
```

- **Parameters:** `pname` (`VERTEX_ATTRIB_ARRAY_ENABLED`, `_SIZE`, `_STRIDE`, `_TYPE`, `_NORMALIZED`, `_BUFFER_BINDING`, `CURRENT_VERTEX_ATTRIB`).
- **Description:** Queries state properties of the attribute at `index` .

`glVertexAttribOffset()`

Spec Declaration:

```
[WebGLHandlesContextLoss] GLintptr glVertexAttribOffset(GLuint index, GLenum pname);
```

- **Description:** Returns the byte offset assigned to attribute pointer at `index` (using `VERTEX_ATTRIB_ARRAY_POINTER`).

Constant Generic Attribute Setters:

- `undefined glVertexAttrib1f(GLuint index, GLfloat x); ()`
- `undefined glVertexAttrib2f(GLuint index, GLfloat x, GLfloat y); ()`
- `undefined glVertexAttrib3f(GLuint index, GLfloat x, GLfloat y, GLfloat z); ()`
- `undefined glVertexAttrib4f(GLuint index, GLfloat x, GLfloat y, GLfloat z, GLfloat w); ()`
- `undefined glVertexAttrib1fv(GLuint index, Float32List values); ()`
- `undefined glVertexAttrib2fv(GLuint index, Float32List values); ()`
- `undefined glVertexAttrib3fv(GLuint index, Float32List values); ()`
- `undefined glVertexAttrib4fv(GLuint index, Float32List values); ()`
 - **Description:** Sets the constant/generic value for the vertex attribute at `index` when vertex attribute arrays are disabled.

4.8 Uniform State (Scalars, Vectors, Matrices)

`getUniformLocation()`

Spec Declaration:

```
WebGLUniformLocation? getUniformLocation(WebGLProgram program, DOMString name);
```

- **Description:** Returns the `WebGLUniformLocation` handle for the uniform `name` in `program`, or `null` if inactive.

`getActiveUniform()`

Spec Declaration:

```
WebGLActiveInfo? getActiveUniform(WebGLProgram program, GLuint index);
```

- **Description:** Returns metadata (`name`, `size`, `type`) for active uniform variable at `index` .

`getUniform()`

Spec Declaration:

any `getUniform(WebGLProgram program, WebGLUniformLocation location);`

- **Description:** Retrieves the current value of the uniform at `location`.

Scalar and Vector Uniform Updaters:

Method Signature	Vector Size	Value Type	Parameter Types
<code>uniform1f(location, x); ()</code>	1	Float	<code>GLfloat</code>
<code>uniform2f(location, x, y); ()</code>	2	Float	<code>GLfloat, GLfloat</code>
<code>uniform3f(location, x, y, z); ()</code>	3	Float	<code>GLfloat, GLfloat, GLfloat</code>
<code>uniform4f(location, x, y, z, w); ()</code>	4	Float	<code>GLfloat, GLfloat, GLfloat, GLfloat</code>
<code>uniform1i(location, x); ()</code>	1	Int/ Sampler	<code>GLint</code>
<code>uniform2i(location, x, y); ()</code>	2	Int	<code>GLint, GLint</code>
<code>uniform3i(location, x, y, z); ()</code>	3	Int	<code>GLint, GLint, GLint</code>
<code>uniform4i(location, x, y, z, w); ()</code>	4	Int	<code>GLint, GLint, GLint, GLint</code>

Overloaded Array Uniform Updaters:

Method Signature	Vector Size	Input List Type
<code>uniform1fv(location, Float32List v); ()</code>	1	<code>Float32List</code>
<code>uniform2fv(location, Float32List v); ()</code>	2	<code>Float32List</code>
<code>uniform3fv(location, Float32List v); ()</code>	3	<code>Float32List</code>
<code>uniform4fv(location, Float32List v); ()</code>	4	<code>Float32List</code>
<code>uniform1iv(location, Int32List v); ()</code>	1	<code>Int32List</code>
<code>uniform2iv(location, Int32List v); ()</code>	2	<code>Int32List</code>
<code>uniform3iv(location, Int32List v); ()</code>	3	<code>Int32List</code>
<code>uniform4iv(location, Int32List v); ()</code>	4	<code>Int32List</code>

Matrix Uniform Updaters:

- `uniformMatrix2fv(WebGLUniformLocation? location, GLboolean transpose, Float32List value); ()`
Uploads a 2x2 float matrix (4 elements). In WebGL 1.0, `transpose` must strictly be `false`.

- `undefined uniformMatrix3fv(WebGLUniformLocation? location, GLboolean transpose, Float32List value); ()`
Uploads a 3x3 float matrix (9 elements). `transpose` must strictly be `false`.
 - `undefined uniformMatrix4fv(WebGLUniformLocation? location, GLboolean transpose, Float32List value); ()`
Uploads a 4x4 float matrix (16 elements). `transpose` must strictly be `false`.
-

4.9 Textures & Samplers Management

`createTexture()`

Spec Declaration:

```
WebGLTexture createTexture();
```

- **Description:** Creates and returns a new uninitialized `WebGLTexture` object.

`bindTexture()`

Spec Declaration:

```
undefined bindTexture(GLenum target, WebGLTexture? texture);
```

- **Parameters:** `target` (`TEXTURE_2D` or `TEXTURE_CUBE_MAP`), `texture` (or `null`).
- **Description:** Binds `texture` to target on the currently active texture unit.

`activeTexture()`

Spec Declaration:

```
undefined activeTexture(GLenum texture);
```

- **Parameters:** `texture` (`TEXTURE0` through `TEXTURE31`).
- **Description:** Selects the active texture unit manipulated by subsequent texture operations.

`deleteTexture()`

Spec Declaration:

```
undefined deleteTexture(WebGLTexture? texture);
```

- **Description:** Marks `texture` for deletion and frees its GPU memory.

`isTexture()`

Spec Declaration:

```
[WebGLHandlesContextLoss] GLboolean isTexture(WebGLTexture? texture);
```

- **Description:** Returns `true` if `texture` is a valid `WebGLTexture` object.

generateMipmap()

Spec Declaration:

```
undefined generateMipmap(GLenum target);
```

- **Parameters:** `target` (`TEXTURE_2D` or `TEXTURE_CUBE_MAP`).
- **Description:** Generates a complete mipmap pyramid for the texture bound to `target` .

texParameteri()

Spec Declaration:

```
undefined texParameteri(GLenum target, GLenum pname, GLint param);
```

texParameterf()

Spec Declaration:

```
undefined texParameterf(GLenum target, GLenum pname, GLfloat param);
```

- **Parameters:**
 - `pname` : `TEXTURE_MAG_FILTER` , `TEXTURE_MIN_FILTER` , `TEXTURE_WRAP_S` , `TEXTURE_WRAP_T` .
 - `param` : Filter constant (`NEAREST` , `LINEAR` , etc.) or wrap constant (`REPEAT` , `CLAMP_TO_EDGE` , `MIRRORED_REPEAT`).
- **Description:** Sets sampling parameters on the texture bound to `target` .

getTexParameter()

Spec Declaration:

```
any getTexParameter(GLenum target, GLenum pname);
```

- **Description:** Returns current texture parameter configuration.

Texture Image Upload Methods:

- **undefined texImage2D(GLenum target, GLint level, GLint internalformat, GLsizei width, GLsizei height, GLint border, GLenum format, GLenum type, [AllowShared] ArrayBufferView? pixels); ()**
Uploads raw pixel memory from a TypedArray into texture at specified mipmap `level` . `border` must be 0.
- **undefined texImage2D(GLenum target, GLint level, GLint internalformat, GLenum format, GLenum type, TextureSource source); ()**
Overloaded browser-convenience method taking DOM elements / canvases / video frames directly (`TextureSource`). May throw `DOMException` .
- **undefined texSubImage2D(GLenum target, GLint level, GLint xoffset, GLint yoffset, GLsizei width, GLsizei height, GLenum format, GLenum type, [AllowShared] ArrayBufferView? pixels); ()**
Replaces a rectangular subregion of an existing texture level using raw pixel data.
- **undefined texSubImage2D(GLenum target, GLint level, GLint xoffset, GLint yoffset, GLenum format, GLenum type, TextureSource source); ()**

Replaces a rectangular subregion of an existing texture level using a `TexImageSource`. May throw `DOMException`.

- `undefined copyTexImage2D(GLenum target, GLint level, GLenum internalformat, GLint x, GLint y, GLsizei width, GLsizei height, GLint border); ()`

Copies pixels directly from the currently bound framebuffer into a 2D texture.

- `undefined copyTexSubImage2D(GLenum target, GLint level, GLint xoffset, GLint yoffset, GLint x, GLint y, GLsizei width, GLsizei height); ()`

Copies a rectangular region from the active framebuffer into an existing texture subregion.

Compressed Textures:

- `undefined compressedTexImage2D(GLenum target, GLint level, GLenum internalformat, GLsizei width, GLsizei height, GLint border, [AllowShared] ArrayBufferView data); ()`

Uploads compressed texture data into the bound texture level.

- `undefined compressedTexSubImage2D(GLenum target, GLint level, GLint xoffset, GLint yoffset, GLsizei width, GLsizei height, GLenum format, [AllowShared] ArrayBufferView data); ()`

Uploads compressed texture subregion data.

4.10 Framebuffer & Renderbuffer Subsystem

`createFramebuffer()`

Spec Declaration:

```
WebGLFramebuffer createFramebuffer();
```

- **Description:** Creates and returns a new `WebGLFramebuffer` object.

`bindFramebuffer()`

Spec Declaration:

```
undefined bindFramebuffer(GLenum target, WebGLFramebuffer? framebuffer);
```

- **Description:** Binds `framebuffer` to target (`FRAMEBUFFER`) for offscreen rendering (or binds `null` for the canvas default buffer).

`deleteFramebuffer()`

Spec Declaration:

```
undefined deleteFramebuffer(WebGLFramebuffer? framebuffer);
```

- **Description:** Deletes the specified framebuffer object.

`isFramebuffer()`

Spec Declaration:

```
[WebGLHandlesContextLoss] GLboolean isFramebuffer(WebGLFramebuffer? framebuffer);
```

- **Description:** Checks validity of `WebGLFramebuffer` .

`checkFramebufferStatus()`

Spec Declaration:

```
[WebGLHandlesContextLoss] GLenum checkFramebufferStatus(GLenum target);
```

- **Description:** Tests completeness of the framebuffer bound to `target` . Returns `FRAMEBUFFER_COMPLETE` if ready for rendering, or failure code (`FRAMEBUFFER_INCOMPLETE_ATTACHMENT` , etc.).

`framebufferTexture2D()`

Spec Declaration:

```
undefined framebufferTexture2D(GLenum target, GLenum attachment, GLenum textarget, WebGLTexture? texture, GLint level);
```

- **Parameters:**
 - `attachment` : `COLOR_ATTACHMENT0` , `DEPTH_ATTACHMENT` , `STENCIL_ATTACHMENT` , `DEPTH_STENCIL_ATTACHMENT` .
 - `textarget` : `TEXTURE_2D` or cube map face (`TEXTURE_CUBE_MAP_POSITIVE_X` , etc.).
 - `level` : Mipmap level to attach (typically 0).
- **Description:** Attaches a texture mipmap image to a framebuffer attachment point.

`framebufferRenderbuffer()`

Spec Declaration:

```
undefined framebufferRenderbuffer(GLenum target, GLenum attachment, GLenum renderbuffertarget, WebGLRenderbuffer? renderbuffer);
```

- **Description:** Attaches a `WebGLRenderbuffer` to a framebuffer attachment point.

`getFramebufferAttachmentParameter()`

Spec Declaration:

```
any getFramebufferAttachmentParameter(GLenum target, GLenum attachment, GLenum pname);
```

- **Description:** Queries attachment parameters (`FRAMEBUFFER_ATTACHMENT_OBJECT_TYPE` , `_OBJECT_NAME` , `_TEXTURE_LEVEL` , `_TEXTURE_CUBE_MAP_FACE`).

`createRenderbuffer()`

Spec Declaration:

```
WebGLRenderbuffer createRenderbuffer();
```

- **Description:** Creates and returns a new `WebGLRenderbuffer` object.

`bindRenderbuffer()`

Spec Declaration:

```
undefined bindRenderbuffer(GLenum target, WebGLRenderbuffer? renderbuffer);
```

- **Description:** Binds `renderbuffer` to `RENDERBUFFER`.

`renderbufferStorage()`

Spec Declaration:

```
undefined renderbufferStorage(GLenum target, GLenum internalformat, GLsizei width, GLsizei height);
```

- **Parameters:** `internalformat` (`RGBA4`, `RGB565`, `RGB5_A1`, `DEPTH_COMPONENT16`, `STENCIL_INDEX8`, `DEPTH_STENCIL`).
- **Description:** Allocates memory storage dimensions and format for a bound renderbuffer.

`deleteRenderbuffer()`

Spec Declaration:

```
undefined deleteRenderbuffer(WebGLRenderbuffer? renderbuffer);
```

- **Description:** Deletes the specified renderbuffer object.

`isRenderbuffer()`

Spec Declaration:

```
[WebGLHandlesContextLoss] GLboolean isRenderbuffer(WebGLRenderbuffer? renderbuffer);
```

- **Description:** Checks validity of `WebGLRenderbuffer`.

`getRenderbufferParameter()`

Spec Declaration:

```
any getRenderbufferParameter(GLenum target, GLenum pname);
```

- **Description:** Queries renderbuffer width, height, internal format, and bit depths (see Section 3.19).

4.11 Drawing Primitives & Execution Control

`drawArrays()`

Spec Declaration:

```
undefined drawArrays(GLenum mode, GLint first, GLsizei count);
```

- **Parameters:**
 - `mode` : Primitive topology (`POINTS` , `LINES` , `LINE_STRIP` , `LINE_LOOP` , `TRIANGLES` , `TRIANGLE_STRIP` , `TRIANGLE_FAN`).
 - `first` : Starting vertex index in array.
 - `count` : Number of vertex indices to render.
- **Description:** Renders primitives sequentially from active vertex attribute arrays without indexing.

`drawElements()`

Spec Declaration:

```
undefined drawElements(GLenum mode, GLsizei count, GLenum type, GLintptr offset);
```

- **Parameters:**
 - `mode` : Primitive topology.
 - `count` : Number of elements to render.
 - `type` : Data type of indices in bound `ELEMENT_ARRAY_BUFFER` (`UNSIGNED_BYTE` , `UNSIGNED_SHORT`).
 - `offset` : Byte offset into the index buffer.
- **Description:** Renders primitives using vertex indices read from the currently bound `ELEMENT_ARRAY_BUFFER` .

`flush()`

Spec Declaration:

```
undefined flush();
```

- **Description:** Empties all command buffers, sending pending GPU commands for execution without blocking until completion.

`finish()`

Spec Declaration:

```
undefined finish();
```

- **Description:** Blocks execution and does not return until all previously issued GPU commands are completely processed.

4.12 Pixel Readback & Storage Modes

`readPixels()`

Spec Declaration:

```
undefined readPixels(GLint x, GLint y, GLsizei width, GLsizei height, GLenum format, GLenum type, [AllowShared] ArrayBufferView? pixels);
```

- **Parameters:**

- `x`, `y` : Lower-left corner of pixel rectangle to read.
 - `width`, `height` : Dimensions of pixel rectangle.
 - `format` : Must match `RGBA` (or `IMPLEMENTATION_COLOR_READ_FORMAT`).
 - `type` : Must match `UNSIGNED_BYTE` (or `IMPLEMENTATION_COLOR_READ_TYPE`).
 - `pixels` : TypedArray destination buffer (`Uint8Array`).
- **Description:** Reads a block of pixel color data from the active framebuffer (or default canvas buffer) into client JavaScript memory.
- pixelStorei()**
- Spec Declaration:**
- ```
undefined pixelStorei(GLenum pname, GLint param);
```
- **Parameters:**
    - `pname` :
      - `PACK_ALIGNMENT` (1, 2, 4, 8)
      - `UNPACK_ALIGNMENT` (1, 2, 4, 8)
      - `UNPACK_FLIP_Y_WEBGL` (boolean)
      - `UNPACK_PREMULTIPLY_ALPHA_WEBGL` (boolean)
      - `UNPACK_COLORSPACE_CONVERSION_WEBGL` ( `NONE` or `BROWSER_DEFAULT_WEBGL` )
    - `param` : Setting integer or boolean value.
  - **Description:** Sets pixel unpack and pack storage modes affecting `readPixels` , `texImage2D` , and `texSubImage2D` .

## 5. Summary Matrix: Categorization & Verification

| Category            | Item Count | Key Enums & Functions                                                                                                                                                                                                      | Notes                              |
|---------------------|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|
| Typedefs            | 3          | <a href="#">TexImageSource</a> , <a href="#">Float32List</a> , <a href="#">Int32List</a>                                                                                                                                   | Full WebIDL typing included        |
| Interfaces & Mixins | 3          | <code>WebGLRenderingContextBase</code> ,<br><code>WebGLRenderingContextOverloads</code> ,<br><code>WebGLRenderingContext</code>                                                                                            | Combined via <code>includes</code> |
| Attributes          | 6          | <a href="#">canvas</a> , <a href="#">drawingBufferWidth</a> , <a href="#">drawingBufferHeight</a> ,<br><a href="#">drawingBufferFormat</a> , <a href="#">drawingBufferColorSpace</a> ,<br><a href="#">unpackColorSpace</a> | Full getters and setters           |
| ClearBufferMask     | 3          | <a href="#">DEPTH_BUFFER_BIT</a> , <a href="#">STENCIL_BUFFER_BIT</a> ,<br><a href="#">COLOR_BUFFER_BIT</a>                                                                                                                | Bitfield flags                     |
| BeginMode           | 7          | <a href="#">POINTS</a> , <a href="#">LINES</a> , <a href="#">LINE_LOOP</a> , <a href="#">LINE_STRIP</a> , <a href="#">TRIANGLES</a> ,<br><a href="#">TRIANGLE_STRIP</a> , <a href="#">TRIANGLE_FAN</a>                     | Primitives                         |

|                                  |     |                                                                                                                                                                                                                                                  |                                         |
|----------------------------------|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| AlphaFunction                    | 8   | <a href="#">NEVER</a> through <a href="#">ALWAYS</a>                                                                                                                                                                                             | Flagged as <i>not supported in ES20</i> |
| Blending                         | 20  | Factors ( <a href="#">SRC_ALPHA</a> , <a href="#">ONE_MINUS_SRC_COLOR</a> ), equations ( <a href="#">FUNC_ADD</a> , <a href="#">FUNC_SUBTRACT</a> ), queries ( <a href="#">BLEND_EQUATION_RGB</a> )                                              | Separate RGB/Alpha                      |
| Buffers                          | 10  | <a href="#">ARRAY_BUFFER</a> , <a href="#">ELEMENT_ARRAY_BUFFER</a> , <a href="#">STATIC_DRAW</a> , <a href="#">DYNAMIC_DRAW</a> , etc.                                                                                                          | Targets, bindings, usage                |
| Cull / Front Face                | 5   | <a href="#">FRONT</a> , <a href="#">BACK</a> , <a href="#">FRONT_AND_BACK</a> , <a href="#">CW</a> , <a href="#">CCW</a>                                                                                                                         | Winding & culling                       |
| Depth & Stencil Functions        | 8   | <a href="#">NEVER</a> through <a href="#">ALWAYS</a>                                                                                                                                                                                             | Comparison tests                        |
| Stencil Ops & Back               | 23  | <a href="#">KEEP</a> , <a href="#">REPLACE</a> , <a href="#">INCR_WRAP</a> , <a href="#">STENCIL_BACK_FUNC</a> (8 back queries)                                                                                                                  | Full front and back state               |
| Capabilities (EnableCap)         | 9   | <a href="#">BLEND</a> , <a href="#">DEPTH_TEST</a> , <a href="#">CULL_FACE</a> , <a href="#">SCISSOR_TEST</a> , <a href="#">DITHER</a> , etc.                                                                                                    | Toggleable pipeline gates               |
| Errors                           | 7   | <a href="#">NO_ERROR</a> , <a href="#">INVALID_ENUM</a> , <a href="#">INVALID_VALUE</a> , <a href="#">INVALID_OPERATION</a> , <a href="#">OUT_OF_MEMORY</a> , <a href="#">INVALID_FRAMEBUFFER_OPERATION</a> , <a href="#">CONTEXT_LOST_WEBGL</a> | Error diagnostics                       |
| Parameter Queries (GetPName)     | 45+ | <a href="#">VIEWPORT</a> , <a href="#">SCISSOR_BOX</a> , <a href="#">MAX_TEXTURE_SIZE</a> , bit depths, ranges                                                                                                                                   | Full context introspection              |
| Hints                            | 4   | <a href="#">GENERATE_MIPMAP_HINT</a> , <a href="#">DONT_CARE</a> , <a href="#">FASTEST</a> , <a href="#">NICEST</a>                                                                                                                              | Optimization hints                      |
| Data Types                       | 7   | <a href="#">BYTE</a> , <a href="#">UNSIGNED_BYTE</a> , <a href="#">SHORT</a> , <a href="#">UNSIGNED_SHORT</a> , <a href="#">INT</a> , <a href="#">UNSIGNED_INT</a> , <a href="#">FLOAT</a>                                                       | Typed array mappings                    |
| Pixel Formats & Types            | 10  | <a href="#">RGB</a> , <a href="#">RGBA</a> , <a href="#">DEPTH_COMPONENT</a> , <a href="#">UNSIGNED_SHORT_4_4_4_4</a> , etc.                                                                                                                     | Format combinations                     |
| Shaders & Programs               | 19  | <a href="#">VERTEX_SHADER</a> , <a href="#">FRAGMENT_SHADER</a> , statuses, limits, language version                                                                                                                                             | Pipeline stages & limits                |
| Shader Precision                 | 6   | <a href="#">LOW_FLOAT</a> , <a href="#">MEDIUM_FLOAT</a> , <a href="#">HIGH_FLOAT</a> , <a href="#">LOW_INT</a> , <a href="#">MEDIUM_INT</a> , <a href="#">HIGH_INT</a>                                                                          | Precision queries                       |
| Uniform Types                    | 15  | <a href="#">FLOAT_VEC2</a> .. <a href="#">FLOAT_VEC4</a> , <a href="#">FLOAT_MAT2</a> .. <a href="#">FLOAT_MAT4</a> , samplers                                                                                                                   | Active uniform descriptors              |
| Vertex Attributes                | 7   | <a href="#">VERTEX_ATTRIB_ARRAY_ENABLED</a> , <a href="#">_SIZE</a> , <a href="#">_STRIDE</a> , <a href="#">_TYPE</a> , etc.                                                                                                                     | Attributes introspection                |
| Texture Target / Units / Filters | 47  | <a href="#">TEXTURE_2D</a> , Cube Map faces, <a href="#">TEXTURE0</a> .. <a href="#">TEXTURE31</a> , wrap modes, min/mag filters                                                                                                                 | Complete texture pipeline               |
| FBO & RBO                        | 25  | <a href="#">FRAMEBUFFER</a> , <a href="#">RENDERBUFFER</a> , attachments, internal formats, completeness states                                                                                                                                  | Offscreen pipeline                      |

|                     |                    |                                                                                                                                                                        |                              |
|---------------------|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|
| WebGL Specific      | 5                  | <a href="#">UNPACK_FLIP_Y_WEBGL</a> ,<br><a href="#">UNPACK_PREMULTIPLY_ALPHA_WEBGL</a> ,<br><a href="#">CONTEXT_LOST_WEBGL</a> , etc.                                 | Browser integrations         |
| Functions & Methods | 98 (138 overloads) | <a href="#">drawArrays()</a> , <a href="#">drawElements()</a> , <a href="#">createBuffer()</a> ,<br><a href="#">useProgram()</a> , <a href="#">texImage2D()</a> , etc. | Every single function mapped |